Instruction Selection & Scheduling via SMT

CS517
Raine Wheary, Christine Lin
GitHub project

1) Introduction

We consider the problem of backend code generation in a compiler. By this point in the pipeline, we can understand the input program as having been reduced to a graph of basic operations in some intermediate representation (IR). The backend must further lower these operations into machine instructions.

For linear control flow (i.e. within a single basic block), there are three major tasks to be solved:

- **Instruction selection:** Modern CPU architectures often have complex macroinstructions encoding more than one basic operation. For example, the x86-64 instruction `mov rax, [rcx + 4*rdx + 28]` involves a left shift, two additions, and a load. Other examples include stack operations, multiple loads and stores (e.g. ARM's `ldp` and `stp`), and fused-multiply-add. A compiler will prefer to emit these instructions when applicable, but it can be a net loss if it means intermediate results are no longer available and need to be recomputed. In the last example, if the address value `rcx + 4*rdx + 28` was used in many places, a compiler should compute this effective address with `lea` and reuse it, rather than doing the calculation inside the `mov`.
- **Instruction scheduling:** Even when two machine instructions could are independent of each other, their order relative to each other can impact the performance of the generated code. We model a pipelined but not out-of-order processor, where a pipeline stall occurs if an instruction's inputs are not available.
- **Register allocation:** Programs in IR form use an unlimited number of virtual registers, but these need to be mapped to a limited number of physical registers on the machine.

Each of these tasks should be understood as a search problem. Classically, the backend is split into three phases, with instruction selection, instruction scheduling, and register allocation being performed in order [1]. However, this architecture has its limitations. When deciding whether to emit a higher-latency macroinstruction, there are situations where the optimal choice depends on register pressure or on the ability of the scheduler to avoid a pipeline stall.

Register allocation is well known to be NP-complete, and is typically solved with heuristic algorithms in practice. We will focus on an integrated algorithm for first two problems via reduction to SMT.

2) Problem statement

We will adopt the convention of coloring words and variables related to the IR (the input) blue and those related to the machine program (the output) red.

2.1) The IR program

We assume there is a fixed finite set of **IR opcodes** C and each opcode $\alpha \in C$ has an associated **arity**.

As input, we start with an **IR program**, which is a sequence P of **IR instructions**. Each IR instruction consists of an opcode and a tuple of arguments matching the opcode's arity. We use N for the length of the program.

$$P := \{P_1, ..., P_N\}$$
$$P_i := \alpha(t_1, ..., t_m), \qquad \alpha \in C, \quad m = \text{arity}(\alpha), \quad 1 \le t_1, ..., t_m < i.$$

The requirement $1 \le t_1, ..., t_m < i$ captures the idea that the input program is a DAG, because each parameter of an IR instruction is the index of some previous instruction. This also means that the first instruction must be nullary.

In addition to the sequence of instructions, a subset of program indices $R \subseteq \{1, ..., N\}$ are designated as **results**. An index $i \in R$ being present in the set means that the result of the instruction P_i must be materialized in the machine program, i.e. it cannot be coalesced into an intermediate computation of a machine instruction.

For each instruction P_i in a given IR program, we can build an **expression tree** (denoted $\text{tree}(P_i)$) by recursively replacing the indices t in the parameter list with the nodes they refer to:

$$\text{tree}(P_i) := \alpha\Big(\text{tree}\Big(P_{t_1}\Big), ..., \text{tree}\Big(P_{t_m}\Big)\Big)$$
$$\textbf{where } P_i = \alpha(t_1, ..., t_m)$$

(Expression trees cannot actually be materialized in an implementation, since they may be exponentially large without sharing, but they are useful concept in the specification of the problem.)

2.2) The machine program

We have a finite set C' of **machine opcodes** $\beta \in C'$ with associated arities. A **machine program** M has a similar structure to the input program: it consists of a sequence $\{M_1, ..., M_K\}$ of **machine instructions**, each of which consists of an opcode and a number of previous argument references that matches the arity.

$$M := \{M_1, ..., M_K\}$$
$$M_j := \beta(r_1, ..., r_m), \qquad \beta \in C', \quad m = \text{arity}(\beta), \quad 1 \le r_1, ..., r_m < j.$$

As a convention, we use variables i and t as indices of IR instructions and j and r for machine instructions.

A machine instruction is thought of as standing in for one or more IR instructions. For example, a 3-ary machine opcode $\text{FMA}(a, b, c)$ might represent the computation $\text{add}(a, \text{mul}(b, c))$. We formalize this with the idea of a **machine definition** D, which associates each n-ary machine opcode β with a definition in terms of a tree of IR instructions with n free variables, written τ.

$$\tau := x_k \qquad \text{(free variable)}$$
$$\mid \alpha(\tau_1, ..., \tau_m) \qquad \alpha \in C, \ m = \text{arity}(\alpha)$$

$$\text{fv}(\tau) := \begin{cases} \{x_k\} & \text{if } \tau = x_k \\ \bigcup_{k=1}^{m} \text{fv}(\tau_k) & \text{if } \tau = \alpha(\tau_1, ..., \tau_m) \end{cases} \qquad \text{(set of free variables of a tree } \tau)$$

$$D_\beta := ((x_1, ..., x_m), \tau) \qquad \beta \in C', \ m = \text{arity}(\beta), \ \text{fv}(\tau) = \{x_1, ..., x_m\}$$

Given a machine definition $D_\beta = ((x_1, ..., x_m), \tau)$, we can apply it to a concrete m-tuple of trees, written as $D_\beta(\tau_1, ..., \tau_m)$, by substituting each x_k for τ_k in the body of the definition τ.

Given a machine program and a collection of machine definitions, we can define an analogous notion of **expression tree** for each instruction M_j.

$$\text{tree}(D, M_j) := D_\beta\Big(\text{tree}\Big(D, M_{r_1}\Big), ..., \text{tree}\Big(D, M_{r_m}\Big)\Big)$$
$$\textbf{where } M_j = \beta(r_1, ..., r_m)$$

For a fixed C, C', D, we say that an instruction M_j in a machine program **models** an IR instruction P_i if $\text{tree}(P_i) = \text{tree}(D, M_j)$. In other words, M_j computes the same expression tree as P_i. (In the instruction selection literature, this relationship is referred to as "tree covering.")

Then we can say that a machine program M **models** an IR program (P, R) if for each designated result $i \in R$ there exists some instruction M_j that models P_i. In other words, every IR instruction designated as a result is computed by some machine instruction. Note that the modeling requirement does not forbid the machine program from containing unnecessary instructions.

Also notice that if an index i is not present in R then there is no requirement that the machine program materialize the value of P_i. This is what permits the compiler to use instructions like $\text{FMA}(a, b, c)$ when the user indicates that they don't need the intermediate computation $\text{mul}(b, c)$ to ultimately be saved in a register.

2.3) Latency calculation

Each machine opcode $\beta \in C'$ has an associated **latency**, which we take to be a positive integer. To model an in-order pipelined CPU, each instruction $M_j = \beta(r_1, ..., r_m)$ has the following life cycle:

- **Decode:** The instruction is fetched and decoded by the CPU. This happens the cycle after the previous instruction M_{j-1} is dispatched.
- **Dispatch:** This happens as soon as the instruction is decoded and all the dependencies $M_{r_1}, ..., M_{r_m}$ have been retired.
- **Retire:** This happens L cycles after the instruction is dispatched, where $L = \text{latency}(\beta)$.

A machine program is considered finished once all instructions have been retired; this determines its **total latency.** In notation:

$$\begin{aligned}
\text{decode}(j) &:= \begin{cases} 0 & \text{if } j = 1 \\ 1 + \text{dispatch}(i-1) & \text{if } j > 1 \end{cases} \\
\text{dispatch}(j) &:= \max\{\text{decode}(j), \text{retire}(r_1), ..., \text{retire}(r_m)\} \\
&\quad \textbf{where } M_j = \beta(r_1, ..., r_m) \\
\text{retire}(j) &:= \text{dispatch}(j) + \text{latency}(\beta) \\
&\quad \textbf{where } M_j = \beta(r_1, ..., r_m) \\
\text{latency}(M) &:= \max\{\text{retire}(j) \mid 1 \le j \le K\} \\
&\quad \textbf{where } M = \{M_1, ..., M_K\}
\end{aligned}$$

2.4) Putting it together

We now have all the pieces to define instruction selection and scheduling as a search problem:

> **Problem.** Given
>
> - A set of IR opcodes C and their arities,
> - An IR program P and set of results R,
> - A set of machine opcodes C' and their arities,
> - A map D_β associating opcodes $\beta \in C'$ with machine definitions,
> - A map $\text{latency}(\beta)$ associating opcodes $\beta \in C'$ with their latency,
>
> Find a machine program M such that M models (P, R) and $\text{latency}(M)$ is minimized.

There is a corresponding decision problem: given the above and a latency bound L, determine whether there is a machine program M such that $\text{latency}(M) \le L$ is NP-hard. We claim that the decision problem is NP-hard, even if the values of $\text{latency}(\beta)$ and L are given in unary.

In our implementation, only P, D, and $\text{latency}(-)$ need to be provided by the user. The sets C and C' and associated arities are automatically deduced, and R is hardcoded to be $\{N\}$, i.e. only the last IR instruction is designated as a result. This is not a significant restriction, because if a larger set $R = \{i_1, ..., i_n\}$ is desired, the user can introduce a new n-ary IR opcode tuple and add the "dummy" instruction $\text{tuple}(i_1, ..., i_n)$ at the end of the program, along with a corresponding machine opcode TUPLE with $D_{\text{TUPLE}}(x_1, ..., x_n) = \text{tuple}(x_1, ..., x_n)$. Since there is no way to compute $\text{tuple}(...)$ except using $\text{TUPLE}(...)$, this forces $P_{i_1}, ..., P_{i_n}$ to be materialized.

3) Design of the reduction to SMT

We use the SMT solver Z3 [2] to approach the decision problem. For a fixed $K \in \mathbb{N}$ (the length of the output program) and $L \in \mathbb{N}$ (the latency bound), we generate a formula φ which is satisfiable iff there is a machine program M of length K such that M models (P, R) and $\text{latency}(M) \le L$.

Z3 has built-in support for algebraic data types (`DatatypeSort`), so we can dynamically build a data type representing a well-formed machine instruction. This lets us encode the variables $M_1, ..., M_K$ directly as variables of φ.

For an instruction $M_j = \beta(r_1, ..., r_m)$, this automatically upholds the constraint $m = \text{arity}(\beta)$, but it does not guarantee that $1 \le r_1, ..., r_m < j$. Instead, this constraint is enforced by a collection of assertions in φ. The indices $r_1, ..., r_m$ as bitvectors (`BV`) of the smallest bit width necessary.

To state that M models (P, R), we introduce another collection of integer-valued variables $m_1, ..., m_K$. The value of such a variable $m_j = i$ encodes the fact that the machine instruction M_j models the IR instruction P_i. It is never useful for a machine program to contain an instruction M_j that does not model any IR instruction; conversely, if it would model two distinct P_i and $P_{i'}$, then these instructions have the same expression tree, and therefore could have been combined using common subexpression elimination at an earlier point in the compiler pipeline. This justifies the assumption that "M_j models P_i" is a one-to-one relationship.

Rather than using bitvectors for the variables $\{m_k\}$, we create another algebraic data type that has exactly N variants with no arguments. This is better for Z3 to handle, because it expresses that the bits of the index are unimportant (unlike machine instruction indices, where we need to perform numeric comparison on bitvectors).

Prior to constructing the formula, we build an index with the following structure. For each $1 \le i \le N$, we record which machine opcodes β are candidates for a machine instruction that models P_i, and for each candidate, we record what the arguments to this machine instruction would need to be. This is done by using unification to match P_i against the definition D_β. As an example, if we had $P_9 = \text{add}(3, 7)$ and $P_7 = \text{mul}(1, 4)$, then the set of candidate machine opcodes for P_9 might look like $\{\text{ADD} \mapsto (3, 7), \text{FMA} \mapsto (3, 1, 4)\}$. This expresses the idea that if $m_j = 9$ at some index j, then either:

- $M_j = \text{ADD}(r_1, r_2)$, with M_{r_1} modeling P_3 and M_{r_2} modeling P_7; in other words, $\left(m_{r_1}, m_{r_2}\right) = (3, 7)$.
- $M_j = \text{FMA}(r_1, r_2, r_3)$, with M_{r_1} modeling P_3, M_{r_2} modeling P_1, and M_{r_3} modeling P_4; in other words, $\left(m_{r_1}, m_{r_2}, m_{r_3}\right) = (3, 1, 4)$.

This gives us a way to succinctly encode the assertion that a particular M_j models P_i in the formula. For each $1 \le j \le K$, we add a conjunct to φ that performs a case analysis on M_j, and depending on what opcode it finds, expresses the appropriate constraint on the possible values of m_j. To ensure that every root is modeled we go through each $i \in R$, and add an assertion to φ the that there is some $1 \le j \le K$ such that $m_j = i$. (The existential quantifier can be expanded to a disjunction of K clauses.)

The final set of variables $\ell_1, ..., \ell_K$ are used to express the latency constraint. The assertion that $\text{latency}(M) \le L$ can be encoded almost verbatim from the definition of latency given above.

4) Scaffolding of the reduction

The final SMT formula can be denoted by

$$\varphi(C, P, R, C', D, \text{latency}(-), L, K; M_1, ..., M_K, m_1, ..., m_K, \ell_1, ..., \ell_K),$$

where the semicolon separates inputs from variables of the formula. To produce a minimal-latency output, we start with $L = 1$ and proceed upwards, checking whether $\varphi(..., L, K; M_1, ..., M_K, m_1, ..., m_K, \ell_1, ..., \ell_K)$ is satisfiable for some $1 \le K \le N$. The optimal output program can be read directly from the valuation of $M_1, ..., M_K$ in the satisfying assignment.

5) Analysis

We evaluate the program on synthetic inputs. Unfortunately, with the reduction as designed, Z3 struggles to check satisfiability beyond input programs of 15-20 instructions. If Rust is installed, the program can be evaluated on test inputs as follows:

```
$ cargo run --release -- test2.txt
trying w/ length 1
trying w/ length 2
trying w/ length 3
trying w/ length 4
0: LOAD_B
1: LOAD_C
2: LOAD_A
3: IMPROVEMENT(2, 0, 1)
total latency: 4
```

6) AI acknowledgement

We did not use AI in the creation of the project.

References

[1] G. S. H. Blindell, “Survey on Instruction Selection: An Extensive and Modern Literature Review.” [Online]. Available: https://arxiv.org/abs/1306.4898

[2] M. Research, “The Z3 Theorem Prover.” [Online]. Available: https://github.com/Z3Prover/z3